

Artificial Intelligence HW1

111550132 張家睿

I. Dataset & GitHub:

https://github.com/LouisChang0126/AI_2026/tree/main/hw1

II. Research Question & Motivation:

As a sound engineer for my church band, I aspire to introduce AI into audio mixing console operations. The first step in this complex task is to verify that AI can recognize the sounds of different instruments. This foundational ability is required before we can further measure and adjust the audio balance, which is why I chose instrument sound classification as the topic for this project. I created an audio dataset and chose to classify the sound of 5 instruments: Drum, Piano, Violin, Acoustic Guitar, and Electric Guitar as the topic.

III. Dataset Documentation:

There are two datasets: **dataset1** and **dataset2**, the first one is the small one, and the second is bigger.

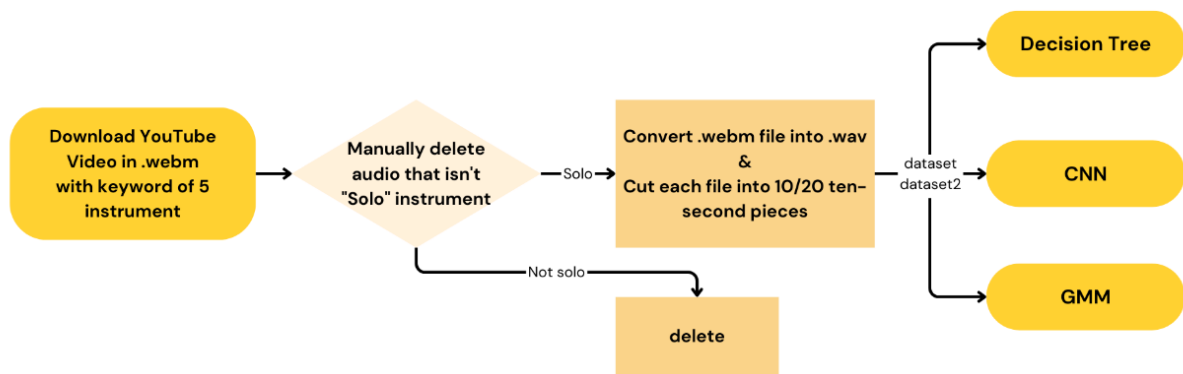
Each dataset contains 5 folders: Drum_solo, Piano_Solo, Violin_Solo, Acoustic_Guitar_Solo, Electric_Guitar_Solo, each folder contains the sound of the instrument, each sound is 10 seconds.

Data type: wav

External source: YouTube

Amount: 437 / 819 of wav file in two datasets, **dataset2** is nearly 2 times larger than **dataset1**.

Data collection: I use 2 Python codes to collect the dataset, the first one is download_youtube_webm.py, which can web crawl from YouTube and download 10 webm files with a specific keyword. After manual checking that the file is actually "Solo" by that instrument, run the second Python file: webm_to_wav.py. It will convert the webm file to wav, cut each file to 10 / 20 10-second pieces, and save the pieces into the dataset folder. (Need to download [ffmpeg](#) for data type converting)



Example: /dataset1/Drum_Solo/ Benny Greb Drum Solo - Drumeo_part2.wav,
/dataset2/Electric_Guitar_Solo/Van Halen - Beat It Solo Cover_part5.wav are two of the data.

IV. Methods Used:

A. Supervised Learning 1: Decision Tree

Public libraries: librosa, numpy, os, sklearn, matplotlib

Feature extractor: use librosa to load and extract the FMCC, ZCR, and RMS from each audio file. And save the mean and std of FMCC, ZCR, and RMS as feature vector.

Data split: 5-fold cross validation (80% train / 20% validate)

Model: by using sklearn, grow a decision tree with a specific depth as an ablation input.

B. Supervised Learning 2: CNN

Public libraries: librosa, numpy, os, sklearn, torch, torchaudio, seaborn, matplotlib, torchvision

Feature extractor: use librosa to load and extract the Mel Spectrogram from each audio and convert them to dB.

Data split: 5-fold cross validation (80% train / 20% validate)

Model: use Pytorch to construct an AudioCNN(without pretrain weight), which has 3 Conv2d layer connected to BatchNorm layer and 2 Fully Connected Layers in the end.

Hyperparameter: batch size = 16, epochs = 10, learning rate = 0.001

C. Supervised Learning 3: SVM

Public libraries: librosa, numpy, os, sklearn, matplotlib

Feature extractor: use librosa to load and extract the FMCC, ZCR, and RMS from each audio file. And save the mean and std of FMCC, ZCR, and RMS as

feature vector.

Data split: 5-fold cross validation (80% train / 20% validate)

Model: by using sklearn, get a SVM classifier with a specific C as an ablation input

Hyperparameter: gamma= 'scale' ($1 / (n_features * X.var())$)

V. Experiments and Results:

A. Supervised Learning 1: Decision Tree

Experiments 1: Is larger dataset better? (compare dataset1 and dataset2)

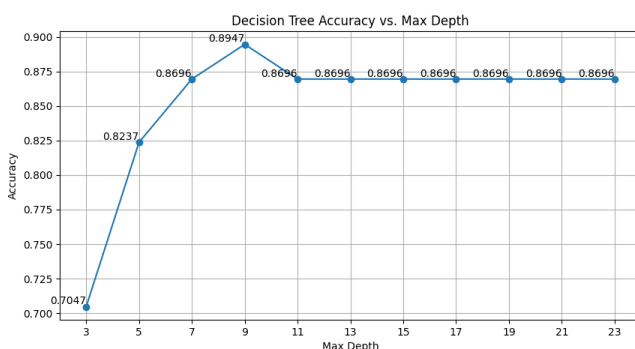
No, by using 5-fold cross validation, small dataset might have better accuracy.

dataset1 (accuracy=0.8947) > dataset2 (accuracy=0.8755) when depth=9. The Decision Tree trained on the larger dataset2 might be forced to memorize newly introduced noise. This can lead to highly fragmented and overly complex decision boundaries, ultimately degrading the model's generalization ability on unseen test data compared to the smaller dataset1.

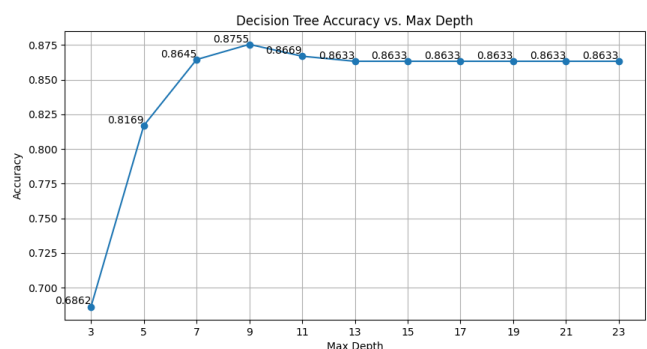
Experiments 2: What is the proper range for tree max_depth?

Consider the grow tree time and the accuracy from two dataset,

$7 < \text{max_depth} < 11$



Acc in different max_depth of dataset1



Acc in different max_depth of dataset2

B. Supervised Learning 2: CNN

Experiments 1: What batch size can get better accuracy?

Batch size	8	16	32	64
dataset1/	0.8673	0.8442	0.8808	0.8925
dataset2/	0.8852	0.9060	0.8926	0.8828

Acc of 2 datasets

On dataset1/, batch size = 64 has the best accuracy.

On dataset2/, batch size = 16 has the best accuracy.

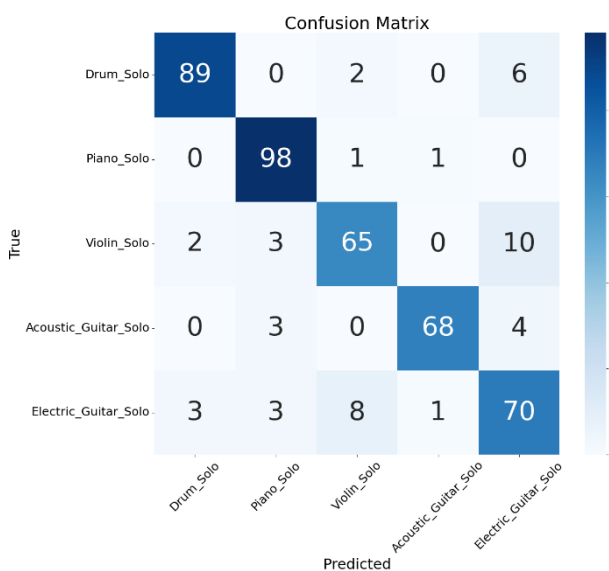
The weight update oscillation caused by a smaller batch size inherently acts as a regularization mechanism, helping the model escape local minima. For the slightly larger and more diverse dataset2, a batch size of 16 introduces an appropriate level of noise that aids in generalization. Conversely, for the extremely small dataset1, a larger batch size provides a more stable gradient descent path, though it increases the risk of simply memorizing the training samples.

In my opinion, dataset2's result is more useful, cause comparing to a normally dataset, both datasets here are small.

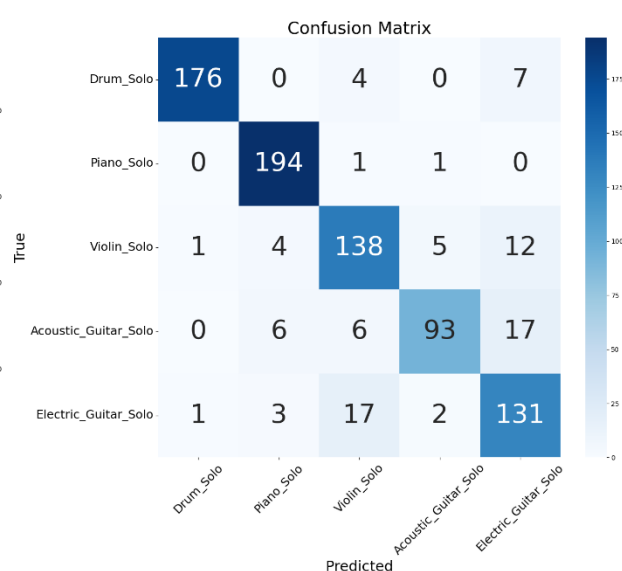
Experiments 2: Is larger dataset better? (compare dataset1 and dataset2)

Yes, by using 5-fold cross validation,

dataset1 (accuracy=0.8442) < dataset2 (accuracy=0.9060) when batch size=16.



Confusion matrix of dataset1



Confusion matrix of dataset2

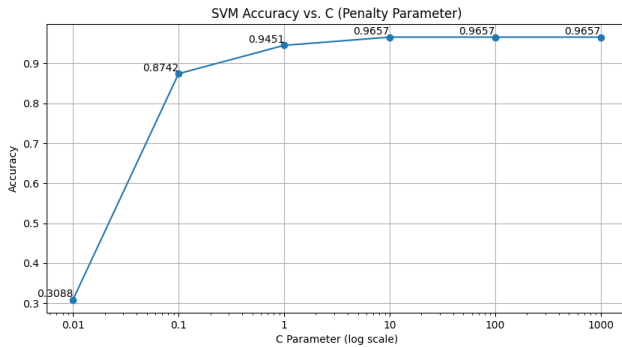
C. Supervised Learning 3: SVM

Experiments 1: Is larger dataset better? (compare dataset1 and dataset2)

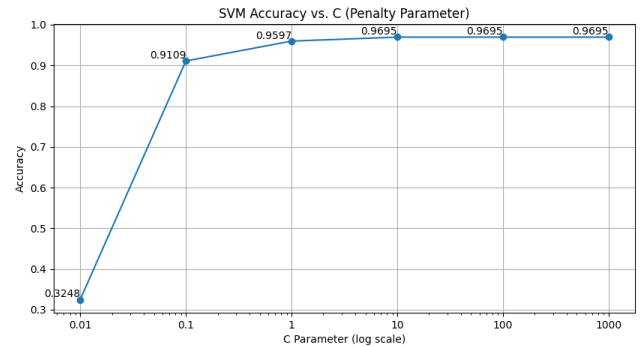
Yes, but not so important when Penalty Parameter (C) is large, by using 5-fold cross validation, dataset1 (accuracy=0.9657) =< dataset2 (accuracy=0.9695) when c=10.

Experiments 2: What is the proper range for Penalty Parameter (c)?

Consider the Penalty Parameter and the accuracy from two dataset,
 $10 < C < 100$. It a little weird cause the curve didn't overfit, and I check by
 changing the gamma value. The curve didn't overfit when $C \geq 1000$ because
 the dataset is small, and the gamma value was fixed at a robust value.



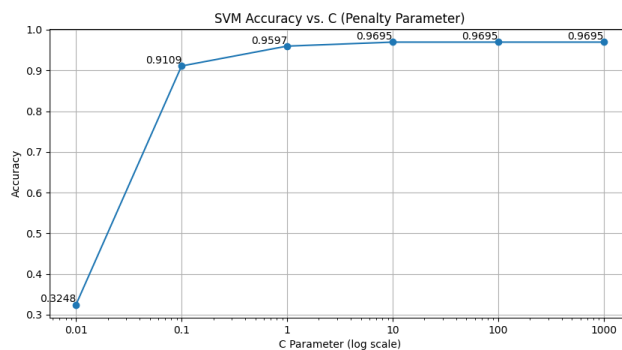
Acc in different Penalty Parameter of dataset1



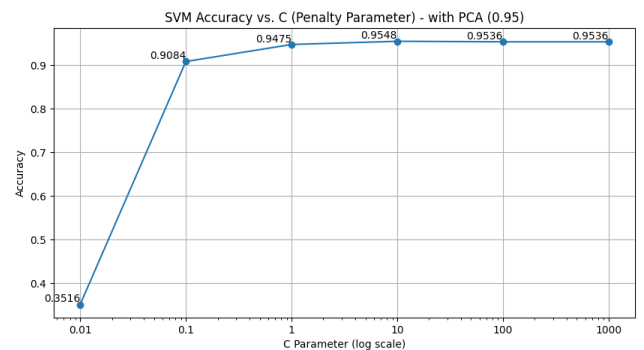
Acc in different Penalty Parameter of dataset2

Experiments 3: Is dimensionality reduction (PCA) helpful for improve accuracy?

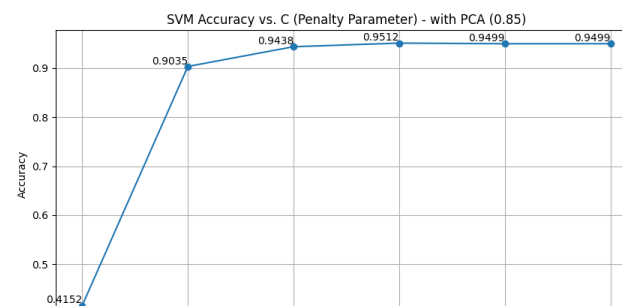
No, when applying PCA to dataset2. The accuracy decreases based on the
 range of the reduction. Maybe is because the dataset information isn't too
 large for the SVM model, the dimensionality reduction only has a
 disadvantage on accuracy.



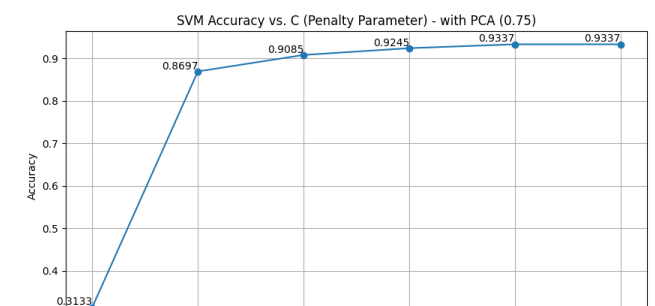
Acc of dataset2 withoto PCA



Acc of dataset2 n_components of PCA=0.95



Acc of dataset2 n_components of PCA=0.85



Acc of dataset2 n_components of PCA=0.75

The extracted features are already highly condensed, low-dimensional representations of the audio. Each statistical feature carries a specific acoustic significance crucial for distinguishing instruments. Applying PCA to these already meaningful, low-dimensional statistics likely discards critical variance and useful classification information, thereby reducing the overall accuracy.

VI. Discussion:

A. Based on your experiments, are the results and observed behaviors what you expect?

In the point of view of accuracy, Decision SVM (0.9657) > CNN (0.9060) > Decision Tree (0.8947). However, by experience, I expect CNN > SVM > Decision Tree. Because I thought CNN is more commonly used and SVM isn't. I believe it is due to the amount of the dataset1, the dataset2 only contains 819 audio files, so the CNN doesn't have enough data to fit nicely.

B. Discuss factors that affect the performance, including dataset characteristics.

- In Decision Tree, the number of max_depth affects the performance a lot. (16%)
- The batch size in CNN affects the performance a little bit. (2~3%)
- In SVM, the value of Penalty Parameter affects the performance a lot. (64~66%). Also, if adjust the gamma value, the performance will have a larger variation.
- By Decision Tree and SVM, large dataset doesn't always have significant improvement.

C. Describe experiments that you would do if there were more time available.

If I have more time, I will try to make a dataset which has more than 10000 audios and test the models again. And I will try to find a pre-trained CNN model which is fit for this experiment.

D. Indicate what you have learned from the experiments as well as your remaining questions.

What I have learned:

This is my first time training a model with audio files. I learned how to extract

features of audio files and how to train audio models in machine learning. I also learned the structure of SVM, the usage of PCA to reduce dimensions. And I break the myth that Deep Learning (CNN) is always better than Traditional Machine Learning (SVM). On a small amount of data, SVM can perform better or similarly to CNN.

Remaining questions:

In the whole experiment, I cut all audio files to 10 seconds to prevent some of the unbalance from the audio time (1 hour vs. 1 minute). But I don't know whether the cutting is better for training the model. If yes, how long of a period is better?

VII. References:

- A. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- B. McFee, B., et al. (2015). librosa: Audio and music signal analysis in python. In Proceedings of the 14th python in science conference.
- C. Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems, 32.
- D. Yeh James. (2017). [資料分析&機器學習] 第 3.4 講：支援向量機(Support Vector Machine)介紹. Medium. <https://medium.com/jameslearningnote/資料分析-機器學習-第 3-4 講-支援向量機-support-vector-machine-介紹-9c6c6925856b>
- E. Papia Nandi. (2021). CNNs for Audio Classification. towards data science. <https://towardsdatascience.com/cnns-for-audio-classification-6244954665ab/>
- F. OpenAI. (2026). ChatGPT. <https://chat.openai.com/>

VIII. Appendix:(All codes are in [GitHub](#).)

Collect dataset:

`download_youtube_webm.py`

```
def download_audio(keyword, count=10, audio_format='mp3'):
    try:
        # Create a folder with the same name as the keyword
        save_dir = keyword.replace(" ", "_")
        os.makedirs(save_dir, exist_ok=True)

        # Search for YouTube video IDs
        search_query = f"ytsearch{count}:{keyword}"
        result = subprocess.run(
            ["yt-dlp", search_query, "--print", "%(id)s"],
            capture_output=True, text=True, check=True
        )

        video_ids = result.stdout.strip().split('\n')

        if not video_ids:
            print("No related videos found.")
            return

        print(f"Found {len(video_ids)} videos, starting audio download...")

        # Download audio
        for video_id in video_ids:
            video_url = f"https://www.youtube.com/watch?v={video_id}"
            print(f"Downloading: {video_url}")
            subprocess.run([
                "yt-dlp", "-x", "--audio-format", audio_format,
                "-o", os.path.join(save_dir, "%(title)s.%(ext)s"),
                video_url
            ])
```

webm_to_wav.py

```

clip_duration = 10 # Duration of each segment in seconds
num_clips = 10 # or 20 # Number of segments per video

def extract_audio_segments(input_folder):
    for file in os.listdir(input_folder):
        if file.endswith(".webm"):
            video_path = os.path.join(input_folder, file)
            output_folder = os.path.join("dataset", input_folder)
            os.makedirs(output_folder, exist_ok=True)
            print(video_path)
            # Extract 20 segments of 10 seconds each from the webm video and
convert to .wav
            filename = os.path.splitext(os.path.basename(video_path))[0]
            print(f"Processing {filename}...")
            # Get the total duration of the video
            cmd_duration = [
                "ffprobe", "-v", "error", "-show_entries", "format=duration",
                "-of", "default=noprint_wrappers=1:nokey=1", video_path
            ]
            result = subprocess.run(cmd_duration, stdout=subprocess.PIPE,
stderr=subprocess.PIPE, text=True)
            try:
                total_duration = float(result.stdout.strip())
            except ValueError:
                print(f"Failed to get duration of {video_path}")
                return

            # Ensure the video is long enough
            if total_duration < clip_duration * num_clips:
                print(f"{video_path} is shorter than {clip_duration * num_clips}
seconds, extracting available segments")

            # Extract audio segments
            for i in range(num_clips):
                start_time = i * clip_duration
                if start_time + clip_duration > total_duration:
                    break # Avoid exceeding video duration

```

```

output_wav = os.path.join(output_folder, f"{filename}_part{i+1}.wav")
cmd_extract = [
    "ffmpeg", "-i", video_path, "-vn", "-acodec", "pcm_s16le", "-ar", "22050",
    "-ac", "1", "-ss", str(start_time), "-t", str(clip_duration), output_wav, "-y"
]
subprocess.run(cmd_extract, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
print(f"Extracted {output_wav}")

```

decision_tree.py

```

# Function to extract audio features from a given file
def extract_features(file_path):
    y, sr = librosa.load(file_path, sr=None)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13) # Extract MFCC features
    zcr = librosa.feature.zero_crossing_rate(y) # Extract ZCR feature
    rms = librosa.feature.rms(y=y) # Extract RMS energy feature

    # Combine extracted features into a single feature vector
    feature_vector = np.hstack([
        np.mean(mfcc, axis=1),
        np.std(mfcc, axis=1),
        np.mean(zcr),
        np.std(zcr),
        np.mean(rms),
        np.std(rms)
    ])
    return feature_vector

def data_prepare(base_path="dataset2/"):
    # Initialize data storage
    data = []
    labels = []

    # class labels
    class_names = ["Drum_Solo", "Piano_Solo", "Violin_Solo", "Acoustic_Guitar_Solo",
"Electric_Guitar_Solo"]

```

```

# Loop through each instrument category
for label, class_folder in enumerate(class_names):
    print(f"Extract features from {class_folder}...")
    class_path = os.path.join(base_path, class_folder)

    # Loop through each audio file in the category
    for file in os.listdir(class_path):
        if file.endswith(".wav"):
            file_path = os.path.join(class_path, file)

            # Extract features from the audio file
            features = extract_features(file_path)

            # Append features and corresponding label
            data.append(features)
            labels.append(label)

data = np.array(data)
labels = np.array(labels)

# Return data and labels without splitting here; cross validation handles splitting
and scaling
return data, labels

def decision_tree(X_train, X_test, y_train, y_test, max_depth=5, random_seed=123):
    # Train a Decision Tree classifier with a maximum depth
    clf = DecisionTreeClassifier(max_depth=max_depth, random_state=random_seed)
    print(f"Growing tree with max depth = {max_depth}...")
    clf.fit(X_train, y_train)

    # Test the trained model on the test dataset
    y_pred = clf.predict(X_test)

```

```

# Calculate and print the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Testing Accuracy: {accuracy:.4f}")
return accuracy

# plot the accuracy vs. max depth
def plot(accuracy):
    plt.figure(figsize=(10, 5))
    plt.plot(accuracy.keys(), accuracy.values(), marker='o', linestyle='-')

    plt.title("Decision Tree Accuracy vs. Max Depth")
    plt.xlabel("Max Depth")
    plt.ylabel("Accuracy")
    plt.xticks(list(accuracy.keys()))

    for key, value in accuracy.items():
        plt.text(key, value, f"{value:.4f}", ha='right', va='bottom')

    plt.grid(True)
    plt.savefig('decision_tree_depth.png')

if __name__ == "__main__":
    data, labels = data_prepare(base_path="dataset1/") # dataset2/

    KFOLDS = 5
    kf = KFold(n_splits=KFOLDS, shuffle=True, random_state=123)

    accuracy = {}
    for i in range(3, 24, 2):
        fold_accs = []
        print(f"--- Evaluating max_depth = {i} ---")
        for fold, (train_idx, val_idx) in enumerate(kf.split(data)):
            print(f"Fold {fold+1}/{KFOLDS}")
            X_train, X_test = data[train_idx], data[val_idx]
            y_train, y_test = labels[train_idx], labels[val_idx]

```

CNN.py

```
# Set device to GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Hyperparameter
SAMPLE_RATE = 22050
N_MELS = 128
BATCH_SIZE = 16
EPOCHS = 10
LEARNING_RATE = 0.001
KFOLDS = 5

# Random seeds
random.seed(123)
np.random.seed(123)
torch.manual_seed(123)
torch.cuda.manual_seed_all(123)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

def extract_mel_spectrogram(file_path):
    # Extracts mel spectrogram from an audio file and convert to decibel scale.
    y, sr = librosa.load(file_path, sr=SAMPLE_RATE)
    mel_spec = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=N_MELS)
    mel_spec = librosa.power_to_db(mel_spec, ref=np.max)
    return mel_spec

class AudioDataset(Dataset):
    # Custom PyTorch dataset for loading and processing audio files.
    def __init__(self, base_path, class_names):
        self.data = []
        self.labels = []
        self.class_to_idx = {cls: idx for idx, cls in enumerate(class_names)} # Map class
names to indices
```

```

# Load data and labels
for class_name in class_names:
    class_path = os.path.join(base_path, class_name)
    for file in os.listdir(class_path):
        if file.endswith(".wav"):
            file_path = os.path.join(class_path, file)

            # Extract features from the audio file
            mel_spec = extract_mel_spectrogram(file_path)

            # Append features and corresponding label
            self.data.append(mel_spec)
            self.labels.append(self.class_to_idx[class_name])

self.data = np.array(self.data)
self.labels = np.array(self.labels)
print(f"Dataset loaded: {len(self.data)} samples")

# Define transformation for input data
self.transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])

def __len__(self):
    return len(self.data)

def __getitem__(self, idx):
    mel_spec = self.transform(self.data[idx]) # Apply transformation
    label = torch.tensor(self.labels[idx], dtype=torch.long)
    return mel_spec, label

class AudioCNN(nn.Module):
    # CNN for audio classification.
    def __init__(self, num_classes):
        super(AudioCNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1)

```

```

self.bn1 = nn.BatchNorm2d(32)
self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
self.bn2 = nn.BatchNorm2d(64)
self.conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1)
self.bn3 = nn.BatchNorm2d(128)
self.pool = nn.MaxPool2d(2, 2)
self.fc1 = nn.Linear(128 * 16 * 53, 512)
self.fc2 = nn.Linear(512, num_classes)
self.relu = nn.ReLU()

def forward(self, x):
    x = self.relu(self.bn1(self.conv1(x)))
    x = self.pool(x)
    x = self.relu(self.bn2(self.conv2(x)))
    x = self.pool(x)
    x = self.relu(self.bn3(self.conv3(x)))
    x = self.pool(x)
    x = x.view(x.size(0), -1)
    x = self.relu(self.fc1(x))
    x = self.fc2(x)
    return x

def plot_confusion_matrix(y_true, y_pred, class_names):
    # Plots a confusion matrix for predictions.
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(14, 12))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names,
yticklabels=class_names)
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.title('Confusion Matrix')
    plt.xticks(rotation=45)
    plt.yticks(rotation=0)
    plt.savefig('confusion_matrix.png')

if __name__ == "__main__":
    # Dataset and class labels

```

```

base_path = "dataset1/" # dataset2
class_names = ["Drum_Solo", "Piano_Solo", "Violin_Solo", "Acoustic_Guitar_Solo",
"Electric_Guitar_Solo"]

dataset = AudioDataset(base_path, class_names)
accs = []
kf = KFold(n_splits=KFOLDS, shuffle=True, random_state=123)

# Cross validation
for fold, (train_idx, val_idx) in enumerate(kf.split(dataset)):
    print(f"Fold {fold+1}/{KFOLDS}")
    train_subset = Subset(dataset, train_idx)
    val_subset = Subset(dataset, val_idx)

    train_loader = DataLoader(train_subset, batch_size=BATCH_SIZE, shuffle=True)
    val_loader = DataLoader(val_subset, batch_size=BATCH_SIZE, shuffle=False)

    model = AudioCNN(num_classes=len(class_names)).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

# Training
for epoch in range(EPOCHS):
    model.train()
    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

# Evaluation
model.eval()
y_true, y_pred = [], []
with torch.no_grad():
    for inputs, labels in val_loader:

```

```

        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        _, predicted = torch.max(outputs, 1)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(predicted.cpu().numpy())

    acc = accuracy_score(y_true, y_pred)
    accs.append(acc)
    print(f"Fold {fold+1} Accuracy: {acc:.4f}")
    plot_confusion_matrix(y_true, y_pred, class_names)

print(f"Total Accuracy: {np.mean(accs):.4f}")

```

SVM.py

```

# Function to extract audio features from a given file
def extract_features(file_path):
    y, sr = librosa.load(file_path, sr=None)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13) # Extract MFCC features
    zcr = librosa.feature.zero_crossing_rate(y) # Extract ZCR feature
    rms = librosa.feature.rms(y=y) # Extract RMS energy feature

    # Combine extracted features into a single feature vector
    feature_vector = np.hstack([
        np.mean(mfcc, axis=1),
        np.std(mfcc, axis=1),
        np.mean(zcr),
        np.std(zcr),
        np.mean(rms),
        np.std(rms)
    ])
    return feature_vector

def data_prepare(base_path="dataset2/"):
    # Initialize data storage
    data = []
    labels = []

```

```

# class labels
class_names = ["Drum_Solo", "Piano_Solo", "Violin_Solo",
"Acoustic_Guitar_Solo", "Electric_Guitar_Solo"]

# Loop through each instrument category
for label, class_folder in enumerate(class_names):
    print(f"Extract features from {class_folder}...")
    class_path = os.path.join(base_path, class_folder)

    # Loop through each audio file in the category
    for file in os.listdir(class_path):
        if file.endswith(".wav"):
            file_path = os.path.join(class_path, file)

            # Extract features from the audio file
            features = extract_features(file_path)

            # Append features and corresponding label
            data.append(features)
            labels.append(label)

data = np.array(data)
labels = np.array(labels)

# Return data and labels without splitting here; cross validation handles splitting
and scaling
return data, labels

def svm_model(X_train, X_test, y_train, y_test, C=1.0, random_seed=123):
    # Train a Support Vector Machine classifier with a specific C
    clf = SVC(C=C, kernel='rbf', random_state=random_seed)
    print(f"Training SVM with C = {C}...")
    clf.fit(X_train, y_train)

    # Test the trained model on the test dataset
    y_pred = clf.predict(X_test)

```

```

# Calculate and print the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Testing Accuracy: {accuracy:.4f}")
return accuracy

# plot the accuracy vs. C parameter
def plot(accuracy, use_pca=False):
    plt.figure(figsize=(10, 5))

    # Extract keys and values
    c_values = list(accuracy.keys())
    c_labels = [str(c) for c in c_values]
    acc_values = list(accuracy.values())

    # Use semilogx since C values are usually on a logarithmic scale
    plt.semilogx(c_values, acc_values, marker='o', linestyle='-')

    title_suffix = f" - with PCA ({N_COMPONENTS})" if use_pca else ""
    plt.title("SVM Accuracy vs. C (Penalty Parameter)" + title_suffix)
    plt.xlabel("C Parameter (log scale)")
    plt.ylabel("Accuracy")
    plt.xticks(c_values, c_labels)

    for key, value in accuracy.items():
        plt.text(key, value, f"{value:.4f}", ha='right', va='bottom')

    plt.grid(True)
    plt.savefig(f'svm_c{"_PCA" if USE_PCA else ""}.png')

if __name__ == "__main__":
    data, labels = data_prepare(base_path="dataset1/") # dataset2/

    KFOLDS = 5
    kf = KFold(n_splits=KFOLDS, shuffle=True, random_state=123)

    USE_PCA = True # Toggle this to True to enable PCA
    N_COMPONENTS = 0.95 # Retain 95% of variance if PCA is used

```

```

accuracy = {}

# A typical list of C values to test
c_list = [0.01, 0.1, 1, 10, 100, 1000]

for c in c_list:
    fold_accs = []
    print(f"--- Evaluating C = {c} ---")
    for fold, (train_idx, val_idx) in enumerate(kf.split(data)):
        print(f"Fold {fold+1}/{KFOLDS}")
        X_train, X_test = data[train_idx], data[val_idx]
        y_train, y_test = labels[train_idx], labels[val_idx]

        # Standardize the features to improve model performance
        scaler = StandardScaler()
        X_train = scaler.fit_transform(X_train)
        X_test = scaler.transform(X_test)

        if USE_PCA:
            # Apply PCA for dimensionality reduction
            pca = PCA(n_components=N_COMPONENTS, random_state=123)
            X_train = pca.fit_transform(X_train)
            X_test = pca.transform(X_test)

        acc = svm_model(X_train, X_test, y_train, y_test, C=c)
        fold_accs.append(acc)

    avg_acc = np.mean(fold_accs)
    accuracy[c] = avg_acc
    print(f"Total Accuracy for C={c}: {avg_acc:.4f}\n")

plot(accuracy, use_pca=USE_PCA)

```